Checkpointed Exploration for DBBench Modification Tasks

Jayanth Vennamreddy

Abstract

This project compares linear SQL execution with checkpointed exploration for DBBench modification tasks. In linear mode, an agent executes mutating SQL directly against a live MySQL database. In checkpointed mode, the system creates a database checkpoint before state-changing SQL, validates the result, and restores the prior state when validation fails. I also implemented a minimal autonomous SQL-agent loop that prompts an OpenAI model with the task, schema, and sample rows, then executes the generated SQL through the same checkpointing substrate. A small evaluation on DBBench dev tasks shows that checkpointing adds overhead when the first SQL action is already correct, but enables recovery when a candidate action mutates the database incorrectly.

1 Goal

DBBench modification tasks require an agent to issue SQL statements such as `INSERT`, `UPDATE`, and `DELETE`. Unlike read-only SQL tasks, these actions change database state. A wrong mutation can poison the session and make later reasoning unreliable. I compare a linear baseline, which executes SQL directly with no undo mechanism, against checkpointed exploration, which checkpoints the database before mutating SQL, validates the result, restores on failure, and retries an alternative action. The goal is not only to generate SQL, but to study how the execution substrate changes the failure mode of database agents.

2 System Design

The prototype uses a `mysql:8` Docker container as the database environment. A task loader parses DBBench JSONL records, extracts the task description, table name, columns, and initial rows, then creates a MySQL database for one task at a time. For robustness, DBBench table columns are loaded as `TEXT`; some benchmark tables contain loose scraped values that do not fit strict MySQL types such as `YEAR`.

The implementation is organized around several small components. `run_linear.py` resets a task database and executes one SQL action directly. `run_checkpoint.py` checkpoints the database, executes one SQL action, validates the result, and optionally restores. `run_explore.py` tries multiple candidate SQL actions, restoring after failed validation before retrying. `run_batch.py` runs linear and checkpointed modes on the same task set and records success, restores, and runtime. `run_autonomous_agent.py` builds a prompt from the DBBench task, schema, and sample rows, calls an OpenAI model to generate a candidate SQL action, and runs that action through checkpointed validation.

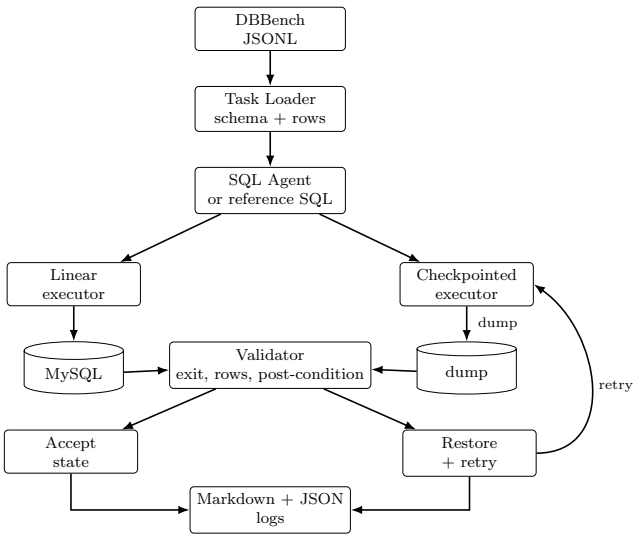

Figure 1: Prototype architecture. The same DBBench task and candidate SQL action can be executed through either the linear baseline or the checkpointed exploration path.

Each run writes both human-readable Markdown logs and structured JSON/JSONL logs. This made it possible to inspect individual failures while also producing aggregate tables. Figure 1 summarizes the execution flow and shows where checkpointing changes the otherwise linear database-agent loop.

3 Checkpointing and Validation

Before state-changing SQL, checkpointed mode creates a dump:

```
mysqldump -uroot -prootpass dbbench >
/tmp/dbbench_checkpoint.sql
```

If validation fails, it restores the database:

```
mysql -uroot -prootpass dbbench <
/tmp/dbbench_checkpoint.sql
```

The current validator checks whether the SQL command returned a successful exit code, whether the number of affected rows matched an expected value, whether the table row-count delta matched the task type, and whether a post-condition query returned the expected row. For simple `INSERT ... VALUES ...` reference statements, the system can infer this post-condition automatically by converting the inserted table, columns, and values into an exact-row `SELECT` query.

For example, for the Madison High School insertion task, the post-condition was:

```
SELECT * FROM ‘School Location Table‘
WHERE ‘School‘ = ’Madison␣High␣School’
  AND ‘Location‘ = ’San␣Diego’;
```

4 Results

Table 1 shows a five-task DBBench dev run using reference SQL actions. In this controlled setting, both modes succeed because the SQL is already correct. Checkpointed mode adds dump overhead, but creates a restore boundary before mutation.

Task	Type	Lin.	Ckpt.	Lin. (s)	Ckpt. (s)
m0	INSERT	pass	pass	0.0858	0.1693
m1	INSERT	pass	pass	0.0767	0.2162
m2	INSERT	pass	pass	0.0773	0.1616
m3	INSERT	pass	pass	0.0716	0.1637
m4	INSERT	pass	pass	0.0656	0.1477

Table 1: Linear vs checkpointed execution on five DBBench dev modification tasks using reference SQL. Checkpointed runs created one checkpoint and performed zero restores in this controlled batch.

In addition to the controlled reference-SQL runs, I tested the autonomous SQL-agent path on the Madison High School insertion task. The model generated the correct `INSERT` statement on the first attempt; checkpointed validation accepted the mutation, and no restore was needed. This verifies that the prompt-generation and model-action loop can plug into the same execution and validation substrate used by the controlled experiments.

5 Case Analysis

Checkpointing helped. In one exploration run, the first candidate inserted `Wrong High School`. The SQL executed successfully and affected one row, so a simple SQL-error check would not catch the mistake. However, the post-condition query for `Madison High School` returned no rows. The system restored the checkpoint and tried the second candidate, which inserted the correct row and passed validation.

Checkpointing did not help. In another run, both candidate SQL actions inserted semantically wrong rows. Checkpointing restored the database after each failed attempt, but no candidate satisfied the task. This shows that checkpointing protects state but does not by itself solve misunderstanding of the task or schema.

Overhead was not worth it. In the reference-SQL batch, the first SQL action was already correct for every task. Checkpointing still succeeded, but added roughly 0.08–0.14 seconds per task from dump overhead. This suggests checkpointing is most useful when actions are uncertain or potentially destructive.

6 Limitations and Next Steps

This prototype uses DBBench reference SQL for the initial batch, so that experiment primarily measures checkpoint overhead rather than full LLM-agent performance. I added a minimal autonomous OpenAI SQL generator and tested it on one task, but a broader evaluation of generated SQL across the full dev set remains future work. The retry experiments still simulate some bad candidate SQL manually in order to isolate the restore-and-retry behavior.

Future improvements include using transaction savepoints when possible instead of full `mysqldump`, restoring by replaying accepted action prefixes, building stronger schema-aware validators, adding safer SQL generation that selects target rows before mutating, and evaluating all DBBench dev modification tasks.

7 Conclusion

The experiment shows that checkpointed exploration changes the failure model for database agents. Linear execution is fast but brittle: once a wrong mutation is applied, the session is corrupted. Checkpointing adds overhead, but creates a recoverable boundary that allows an agent to inspect, reject, restore, and retry. The main research challenge is not only checkpoint creation, but deciding when a mutation should be accepted.